POLICYPILOT

A Permission-Aware, Evaluated Enterprise Knowledge Assistant

Revised Project Proposal

Prepared by Abdul Rauf | October 16, 2026

1. Executive Summary

PolicyPilot is a retrieval-augmented generation (RAG) assistant that answers employee questions using internal-style company documents and provides citations to supporting sources. Its central design goal is to ensure that retrieved information is authorized for the requesting user before it is supplied to the language model. The project will combine hybrid search, reranking, evidence-based answer verification, abstention for insufficiently supported questions, and a reproducible evaluation framework.

The project will be implemented as a production-style portfolio service with a web interface, API, automated tests, containerized deployment, tracing, and documented measurements of answer quality, access-control behavior, latency, and cost. Security claims will be scoped to the implemented controls and tested scenarios; passing the test suite will be reported as evidence of observed behavior, not as proof of universal security.

The project has two goals: to demonstrate a complete and defensible RAG engineering workflow, and to publish a reproducible artifact that explains design decisions, measured trade-offs, limitations, and failure cases.

2. Problem Statement

Organizations maintain policies, handbooks, and procedures across many documents. Employees may spend substantial time locating relevant information, while document-based chatbots can introduce several risks:

- Unsupported or incorrect answers when the retrieved evidence does not justify a response.
- Unauthorized disclosure when access permissions are missing, incorrectly applied, or bypassed.
- Poorly grounded citations that do not support the claims they accompany.
- Unmeasured engineering changes, where new chunking strategies, models, or prompts are adopted without reliable comparison.
- Prompt-injection attempts embedded in retrieved documents that try to influence the model's behavior.

PolicyPilot will address these risks through server-enforced authorization, evidence-linked answer generation, explicit abstention behavior, adversarial testing, and controlled experiments.

3. Objectives

- Build an end-to-end RAG system over a realistic corpus with explicit document and user permissions.
- Apply authorization on the server and filter unauthorized content before it enters the language-model context.
- Compare chunking, keyword search, vector search, hybrid retrieval, and reranking using a fixed evaluation protocol.
- Measure unsupported claims and citation quality before and after an evidence-verification stage.
- Calibrate abstention behavior using answerable and unanswerable questions, while reporting both false answers and unnecessary refusals.
- Evaluate prompt-injection defences against a documented set of attack categories and report observed attack success rates.
- Deliver a reproducible service with automated regression tests, request tracing, and latency and cost reporting.

4. Scope

4.1 In scope

- Document ingestion, parsing, heading-aware chunking, metadata management, and indexing.
- Hybrid retrieval with a cross-encoder reranking stage.
- Server-side role and permission checks, including negative authorization tests.
- Cited answer generation, claim-level evidence checks, abstention, and a basic escalation status.
- A labelled evaluation set, retrieval and answer-quality metrics, and security test scenarios.
- A simple web interface, API, Docker-based deployment, CI checks, and operational measurements.

4.2 Out of scope

- Fine-tuning a language model or training a custom embedding model.
- Integration with real enterprise identity providers.
- Production certification, formal security assurance, and multi-tenant hardening.
- A full human case-management workflow for escalations.

4.3 Possible extensions

- Urdu or bilingual queries.
- Agentic retrieval that decides when another search is needed.
- Table-aware ingestion and support for complex scanned documents.

5. Proposed Solution

5.1 Data and permission model

The initial corpus will use publicly available company handbook material, such as the GitLab Handbook, only after its licence and redistribution terms have been checked. The target is approximately 200–500 selected documents or sections, subject to source availability and processing effort. Synthetic visibility labels will be assigned for testing; these labels will not be represented as the source organisation's actual confidentiality classifications.

The initial access categories will be Public, Employee, Manager, HR, and Finance. These categories will be treated as separate permissions rather than a single hierarchy. A user may have one or more explicitly assigned permissions. Access will be denied by default if identity or permission resolution fails. The authorization matrix will be documented and tested before evaluation.

Example user group	Public	Employee	Manager	HR	Finance
Employee	Allow	Allow	Deny	Deny	Deny
Manager	Allow	Allow	Allow	Deny	Deny
HR staff	Allow	Allow	Deny	Allow	Deny
Finance staff	Allow	Allow	Deny	Deny	Allow

This matrix is an illustrative project policy. Any additional roles, administrators, or multi-role users will be defined explicitly rather than inferred from role names.

5.2 Ingestion pipeline

- Collect approved source documents and record source URL, retrieval date, licence notes, and document version.

- Parse content and preserve headings, section structure, and source references.
- Split documents into heading-aware chunks; document the chunk size, overlap, and any exclusions.
- Attach stable document and chunk IDs, source URL, section, version, update timestamp, and allowed permissions.
- Generate embeddings and index the chunks in the selected search database.
- Record ingestion failures and ensure updated or deleted documents do not leave stale indexed content.
- Validate permission metadata and citation links before marking an ingestion run successful.

5.3 Query and answer pipeline

- Authenticate the demo user and resolve permissions on the server; do not trust a role supplied by the client.
- Run keyword and vector retrieval with authorization filters applied in the retrieval query.
- Recheck authorization before retrieved passages are assembled into the language-model context.
- Rerank the permitted candidates with a cross-encoder and select a fixed, documented number of passages.
- Generate an answer with citations linked to the retrieved document and section identifiers.
- Split the answer into factual claims and check each claim against its cited evidence.
- Classify claims as supported, contradicted, or insufficiently supported; remove or qualify unsupported claims, or abstain when evidence is inadequate.
- Return a clear answer, citations, or an abstention/escalation status. Do not expose restricted passages in errors, citations, logs, or cached responses.

The verifier is a quality-control stage, not a security boundary. Authorization must be enforced independently. Retrieved documents will be treated as untrusted content and must not override system policy or authorize tool actions.

5.4 Initial technology stack

Layer	Initial choice / decision
Language and API	Python and FastAPI
Search database	Choose one database with hybrid-search support (Weaviate or Qdrant) before implementation
Embeddings and reranker	Select one open-source embedding model and one cross-encoder; record exact model identifiers
Generation and judging	Hosted LLM API; record model identifiers, prompts, and relevant parameters
Interface	Streamlit or a lightweight web front end; choose one for the MVP
Operations	Docker Compose, GitHub Actions, structured request metrics, and a tracing tool

The initial configuration will be frozen for baseline measurements. Component changes will be documented in a decision log and evaluated separately where practical. Exact versions and model identifiers will be recorded in the repository so another engineer can reproduce the setup.

6. Methodology and Timeline

The target is four weeks. The effort estimate is approximately 60–80 focused hours; this requires an average of 15–20 hours per week, so the daily schedule must be planned accordingly. If only 2–3 hours are available

on five days each week, the available time will be approximately 40–60 hours and optional deliverables may need to be deferred.

Week	Focus	Key outputs and completion criteria
1	Baseline and data	Approved corpus and licence notes; permission matrix; ingestion pipeline; baseline RAG; initial labelled benchmark and baseline metrics.
2	Retrieval quality	Controlled comparisons of chunking, BM25, vector and hybrid retrieval; reranker comparison; results table including latency.
3	Authorization and answer reliability	Server-side permission filtering; negative access tests; prompt-injection scenarios; claim verification; abstention calibration.
4	Final evaluation and delivery	Frozen evaluation run; CI regression subset; Docker setup; tracing, cost and latency report; README, demo and limitations.

Each major change will be introduced and measured against a fixed evaluation protocol. If the schedule becomes constrained, the priority order will be authorization correctness, reproducible evaluation, reliable answer behavior, deployment, and then presentation polish.

7. Evaluation Plan

The initial evaluation set will contain approximately 60 labelled questions, including answerable and unanswerable questions, exact-term queries, questions with multiple relevant passages, and role-restricted questions. Each item will record its expected answer or abstention outcome, relevant source passages, permitted roles, and category. The 60-question set is a small development benchmark; conclusions will be described as preliminary unless supported by additional independent tests.

Where feasible, questions used to tune thresholds or prompts will be separated from a held-out final evaluation set. If the available dataset is too small for a reliable split, this limitation will be stated explicitly and the results will not be presented as an unbiased estimate of general performance.

Metric	Method	Purpose / reporting
Recall@5 and MRR	Compare retrieved passages with labelled relevant passages	Measure retrieval coverage and ranking quality.
Precision@k / context sufficiency	Assess relevance and whether retrieved evidence is enough to answer	Detect irrelevant or incomplete context.
Answer correctness	Compare responses with labelled answers; manually review a sample	Measure whether final answers are correct.
Unsupported claim rate	Manually reviewed claim labels, with LLM assistance only as a secondary aid	Compare answers before and after verification.
Citation correctness	Check whether cited passages support the claims they accompany	Measure evidence traceability.
Leakage tests	Attempt direct and indirect access across defined roles, including cache and citation cases	Report observed leaks and total tests; zero observed leaks is not proof of universal security.
Abstention precision and coverage	Evaluate answerable and unanswerable questions across thresholds	Show the trade-off between unsupported answers and unnecessary refusals.
Injection success rate	Test documented attack categories against baseline and defence	Report successful attacks, total attempts, and attack types.

	configurations	
Latency and cost	Request logs under a documented test setup	Report p50/p95 latency and estimated cost per 1,000 queries.

7.1 Experimental controls and reproducibility

- Record the baseline chunking, retrieval configuration, model identifiers, prompts, and generation parameters.
- Change one major variable at a time where practical, and keep the evaluation questions and other relevant settings fixed.
- Keep a frozen final evaluation set that is not repeatedly used to tune the system.
- Manually review a sample of answer, citation, and verifier labels; disclose where automated judging is used.
- Repeat selected runs when model variability or infrastructure noise may affect conclusions.
- Report counts alongside percentages and include confidence intervals where appropriate; avoid broad claims from small samples.
- Publish negative results, known failure cases, and quality/latency/cost trade-offs alongside improvements.

8. Deliverables

- A reproducible RAG application with a simple interface and demo credentials, using synthetic role assignments and approved public content.
- A public repository with setup instructions, architecture diagram, evaluation results, and a section explaining failures and changes.
- A decision log linking major design choices to measurements.
- Automated unit, integration, authorization, and regression tests, with a subset of evaluation checks in CI.
- A report covering retrieval, answer quality, citation quality, observed leakage tests, injection tests, latency, cost, and limitations.
- A short walkthrough video and written article if time permits after the core system and evaluation are complete.

9. Risks and Mitigations

Risk	Mitigation
Scope creep	Prioritize authorization and evaluation; defer optional polish and extensions.
Weak or biased labels	Use a documented labelling guide, manually review a sample, and disclose dataset limitations.
LLM-judge bias or variability	Use human-reviewed examples, report the judge model and prompt, and do not treat the judge as ground truth.
Access-control failures	Deny by default; enforce permissions server-side; test retrieval, context assembly, citations, caches, and errors.
Prompt injection	Treat retrieved content as untrusted data; restrict tools; test multiple attack categories and report observed outcomes.
API cost overruns	Set usage limits, cache embeddings where appropriate, use smaller evaluation models where suitable, and cap experiment runs.
Corpus licensing	Confirm licence and redistribution terms; retain attribution and publish processed data only where permitted.
Secrets or sensitive logs exposed	Keep keys out of source control, minimize sensitive logs,

	redact where appropriate, and document retention.
Inconsistent results	Freeze the baseline, version prompts and models, record configuration, and repeat selected experiments.

10. Success Criteria

The following criteria will be evaluated against the documented test setup. Numerical targets that depend on the baseline will be finalized before the final evaluation run rather than selected after results are observed.

- Authorization tests show no observed restricted-content leakage across the defined role, direct-access, citation, and cache test suite; the number and coverage of tests are reported.
- At least one retrieval change produces a measurable improvement over the baseline under the fixed evaluation protocol, or the report explains why no reliable improvement was observed.
- The verifier's effect on unsupported claims is measured before and after verification, together with false rejections, latency, and cost.
- Abstention results report the trade-off between rejecting unanswerable questions and answering answerable questions.
- Prompt-injection results include the attack categories, total attempts, observed successes, and limitations.
- The application starts through documented commands, automated tests run in CI, and a demo is available where hosting permits.
- Another engineer can reproduce the baseline and final evaluation from the repository instructions and recorded configuration.

11. Expected Outcomes and Value

PolicyPilot will produce a reproducible demonstration of practical RAG engineering, with particular emphasis on permission-aware retrieval, evidence-linked answers, evaluation discipline, security testing, and operational reporting. The project will show not only what configuration performed best, but also what failed, how results were measured, and which trade-offs remain unresolved.

The system is a portfolio and learning demonstration. Passing the project's tests will provide evidence about the defined scenarios but will not establish that the system is production-certified or secure against all attacks.

12. Resources Required

- A development machine with Docker installed.
- Access to a hosted LLM API and a controlled budget for evaluation runs.
- A free or low-cost hosting option for the demo, subject to security and cost constraints.
- Approximately 60–80 focused hours over four weeks, or a reduced scope if the available time is closer to 40–60 hours.

Appendix A. Minimum Security Test Scenarios

The initial security test suite should include, at minimum, the following scenario types. The final number of test cases will be recorded in the results.

- A user requests a restricted document by its exact title.
- A user asks an indirect question intended to reveal information from a restricted source.
- A permitted answer contains citations; verify that every cited source is authorized for that user.
- A cached result is requested by a user with a different permission set.
- A user's permission is removed and subsequent requests are tested for stale access.
- A retrieved document contains instructions to override system policy or reveal secrets.
- A document attempts to induce fabricated evidence or misleading citations.
- Authentication or permission resolution fails; the system must deny access safely.

Appendix B. Reporting Template

Configuration	Retrieval metrics	Answer / security metrics	Latency / cost	Notes
Baseline	Recall@5, MRR, Precision@k	Correctness, unsupported claims, citations, observed leaks	p50, p95, cost/1,000	Model and configuration IDs
Hybrid retrieval	Same metrics and dataset	Same answer evaluation where applicable	Same test setup	Difference from baseline
Reranking	Same metrics and dataset	Same answer evaluation where applicable	Same test setup	Incremental benefit
Verification + abstention	Use same retrieved evidence where comparing verifier	Unsupported claims, false rejections, citation correctness	Same test setup	Failure cases and trade-offs